

Sycamore

Independent review pack — everything needed to check this work without trusting it.

Prepared 16 September 2026 **Commit** 9c28a07 **Branch** claude/sycamore-prompts-build-chain-o5rqtu
Repository ssb22inc/New-skills-

Read this first. This document is written to be disbelieved. Every claim in it names the command that checks it, and the sections that matter most are the ones describing what was *wrong*. A reviewer who only reads one section should read §6, Defects found and fixed — including one that would have let a single order pay a seller five times over.

1. What Sycamore is

WhatsApp-first commerce for Caribbean micro-entrepreneurs. A seller runs their business from chat or a voice note: catalogue, bookings, capacity, payments, reviews and payouts. A buyer arrives from a link, books, pays and leaves. There is no app to download and no dashboard anyone is required to use.

Seven product laws govern it. The ones that matter to a reviewer:

- **One door.** Every action can start as a chat message or a voice note. No feature may *require* a dashboard, form, or app.
- **Plain-number ledgers.** Money is reported as "you spent X, it brought in Y", never as a chart the seller must interpret.
- **Show-me-why.** Every automated decision explains itself in one tap.
- **Trust is never traded.** No suppressed reviews, no fabricated imagery, no undisclosed AI voice, no personal data to unvetted vendors.
- **Hold the trust, never the float.** Licensed partners custody money; Sycamore owns the split logic, the ledger and the data.

Money is always integer minor units and a currency code. The ledger is double-entry and append-only. Every table carries a market identifier and every query is scoped by it.

2. How to verify everything in this document

A reviewer with the repository and Docker can reproduce every number here. These are the only commands needed.

```

git clone <repo> && git checkout claude/sycamore-prompts-build-chain-o5rqtu
pnpm install --frozen-lockfile
docker compose up -d                                # Postgres 16 + Redis 7

export DATABASE_URL=postgres://sycamore:sycamore@localhost:5432/sycamore
export REDIS_URL=redis://localhost:6379
export SYCAMORE_REQUIRE_DB=1                        # makes skipped gates a FAILURE

pnpm lint && pnpm format && pnpm typecheck
pnpm test                                           # 295 tests, 0 skipped
pnpm test:coverage                                # core coverage
pnpm --filter @sycamore/tests perf:trust           # trust-page budget
pnpm --filter @sycamore/tests load:profiles         # four load profiles
pnpm --filter @sycamore/tests deploy:audit         # 18 checks against the live origin

```

The SYCAMORE_REQUIRE_DB flag exists because of a real incident: thirty-five test files skip when Postgres is unreachable, so a database that failed to start turned every money gate into "skipped" and the build went green. With the flag set, an unreachable database fails the build. Without it, exactly one test skips by design — the guard that enforces this.

3. Repository map

Path	What lives there	Files
/core	All business logic. Imports no vendor SDK.	115 TypeScript files, 8,878 lines
/packs	Country and industry configuration as YAML data	14 country, 2 industry
/adapters	Every external vendor: payments, channels, media, language models	6
/apps/web	Trust pages, seller's day, chat door, cockpit, developer console	12 routes
/apps/gateway	Webhook ingress, signature verification, queueing	—
/tests	Cross-cutting gates: money, chaos, drills, adversarial, scope	—
test code overall	Across every package	68 files, 10,521 lines

Test code exceeds production code in the core by a fifth. The build specification requires testing to be roughly thirty percent of effort; by line count it is more than half.

The globalization rule, which a reviewer can check mechanically. A new country is one configuration file. A new industry is one configuration file. A new payment provider is one adapter. If core code must change to enter a new market, the abstraction is wrong. This is enforced: the difference between /core at the Jamaica-only commit and at the Jamaica-plus-Dominican-Republic commit must be empty, and continuous integration fails if it is not.

4. Scope and completeness

The build follows a written chain of 37 prompts, P0 through P36, each with an explicit gate. All 37 carry a gate commit. The chain defines no P37; the file ends at P36.

Area	State
Prompt chain P0–P36	37 of 37 gated
Adversarial suite (spec §5.7)	6 of 7 machine-checkable classes; the seventh needs live sellers
Tests	295 passing, 0 skipped with the database-required flag
Container image	Builds and runs; migrates and seeds from an empty database
Deployed origin	Audited in a real browser, 18 of 18 checks
Continuous integration	BLOCKED — see §8
Human gates	10 open; none can be closed by code

What "complete" does and does not mean here. Everything a machine can build without credentials or live sellers is built and tested. No real seller has ever used the system. No real money has moved. Ten gates need a payment partner agreement, WhatsApp Business verification, advertising accounts, counsel sign-off per island, ten Kingston sellers onboarded by voice note alone, and one person tapping "Add to home screen" on a phone.

5. What the tests actually prove

Test counts are worthless without knowing what each one holds down. These are the gates that would matter in a due-diligence review.

Gate	What it proves
Oversell storm	500 concurrent booking attempts on 12 seats produce exactly 12 confirmations. Permanent.
Ledger fuzz	10,000 randomised book/cancel/refund/dispute/retry sequences reconcile to the cent.
Ledger concurrency	A 200-way stampede on one key produces one transaction; 100 parallel orders are exact; release racing refund settles once.
Prompt injection	50 attack prompts against the conversation layer produce zero unauthorised tool calls. User text is data, never instructions.
Review fraud	Four red-team personas — no-booking, burst ring, competitor one-star, device cluster — all refused or held.
Blackout drill	48 simulated hours offline: orders land over SMS, a double-delivered backlog applies exactly once, the ledger balances.
Eviction drill	WhatsApp removed entirely: identities rebind to another channel, 80% of daily flow recovers against a 70% target.
Market isolation	All 13 dark markets' configuration corrupted; the live market is unaffected.
Trust-page budget	Under 100 KB and interactive under 2 seconds on throttled 3G. Measured at 2,978 bytes and 445 ms locally; 1,973 bytes and 532 ms over the wire.
Load profiles	Four traffic shapes including a seller going 100× viral: 2,500 messages at 500/second, zero drops.
No hardcoded copy	No user-facing sentence exists in code; all text resolves through the localization engine or the build fails.
No ad-hoc colour	A raw hex value anywhere in an app surface fails the build. It caught a violation during this session.

6. Defects found and fixed in this session

This is the section to read closely. Each defect was reproduced before being fixed and re-verified after. Three were found by an adversarial review of work completed an hour earlier.

CRITICAL One order could be settled many times

The ledger's release path read "has this order been released?" and then acted on the answer, with no lock between the two. Eight concurrent releases of a single 900,000 capture paid out **4,500,000**. Refunds had the identical shape and could drain escrow past the capture.

Idempotency keys did not help. They stop the *same* key twice; the callers that collide here carry *different* keys by design — a cancellation refund, a dispute refund, a hurricane sweep and the payment-provider webhook are four distinct keys for one order. A hurricane sweep firing while a dispute resolves is two of them at the same instant.

What made it invisible: every posting is internally balanced, so the trial balance stayed level while the seller was credited eight times. No downstream check would have noticed.

Fixed twice over. An exclusive row lock is taken on everything already posted against the order before its sums are read, and a database migration adds a partial unique index so a second release is impossible even if a future caller forgets the lock. Verified by a test that fires eight concurrent releases with distinct keys and asserts exactly one succeeds.

CRITICAL A gate that could not fail

The pre-existing test named "release racing refund on one order never double-settles" fired all three releases under a single idempotency key, then asserted at most one transaction per key. That is precisely what the database's unique index guarantees on its own, whatever the application code does. The assertion could not fail. It now uses distinct keys and asserts one release per *order*.

This is the same failure mode the project had already written a guard against at a different level, committed by the money gate itself.

HIGH Money could change currency between capture and payout

A capture in Jamaican dollars could be refunded or released in US dollars. The balance check sums minor units across currencies with no grouping, so the books would have read level while the money was wrong. Escrow now leaves in the currency it arrived in.

HIGH A split could pay the platform more than came in

The split calculator checked that the four shares summed to 10,000 basis points and nothing else. A seller share of -1,000 with a platform share of 11,000 sums correctly, passed, and returned the seller -10,000 with the platform taking 110,000 out of a 100,000 capture. Shares must now be non-negative whole numbers, checked before the sum. Separately, amounts above the range where floating-point integers stay exact produced arithmetic that overshot its own floor; such amounts are now refused rather than silently mis-split.

HIGH Arbitrary file read through configuration identifiers

Country and industry identifiers name files on disk and had no sanitisation. An identifier of `../../pnpm-workspace` resolved inside the repository, was read and was parsed, surviving until schema validation rejected its shape. Reading

the file at all is the defect. The sink is live: the public trust page passes the market segment straight from the URL to the loader. Identifiers are now validated by shape before becoming a path.

HIGH Continuous integration was red for eight days

Ten consecutive runs failed, each 26 seconds in, at the same step. The cause was a package-manager version declared in two places at once, added so a hosting provider would detect it. Every step after it was skipped, so the suite had not run in continuous integration since 9 September. Every report of "tests green" in that period was a local result on one machine — the exact distinction the database-required guard exists to destroy, reappearing one level up in the pipeline rather than in the tests.

MEDIUM A migration racing a platform freeze

The application migrated its database at boot. On a serverless host an instance can be frozen the moment it answers the request that woke it, so a migration begun at boot was cut off mid-transaction and rolled back — leaving two bookkeeping tables, no schema, and not one line in the log. Every retry reproduced it exactly. Boot now *asks* which migrations are pending in one read-only query and reports a schema that is behind; it applies them only where the process outlives the request.

7. The adversarial review

The specification requires an adversarial suite before every phase gate, naming five attack classes. Two existed. Three did not — split manipulation, stolen-card and chargeback patterns, and web-application attacks with authentication fuzzing. A suite with holes is worse than a missing one, because it reports green over the part nobody wrote.

Twenty-eight attacks now run on every commit, each aimed at behaviour that already existed:

- Forged and malformed webhook signatures, including a byte-flip sweep of the signed body
- SQL and script payloads through a seller's business name, checked in the rendered page
- A Jamaican identifier offered to a Dominican-scoped service
- Configuration identifiers climbing out of their directory
- Refunds exceeding a capture, whole and sliced, sequential and concurrent
- A chargeback landing after the seller has been paid
- Forged ledger entries, and direct UPDATE and DELETE against the append-only trigger using a raw database handle rather than the service

The suite was then itself reviewed adversarially by a separate model briefed to find vacuous tests and unverifiable claims. It found both. The important findings were not in the new tests — they were in the ledger those tests had just marked as passing, and they are the two critical defects in §6. It also found that the first version of the path-traversal test proved nothing, that the fuzz used unseeded randomness so a failure could not be reproduced, that the split tests were gated on a database despite being pure arithmetic, and that a claim of "0 skipped" was true only with a flag the claim did not name.

A reviewer should treat this as the most useful signal in the pack: the second reading found real money defects in code that had just passed a first reading. The right conclusion is not that the suite is now complete. It is that adversarial review found things, and should be repeated.

8. Open items a reviewer should press on

Continuous integration cannot go green as configured

The project lives on a branch. The repository's default branch carries an entirely different product, with no common ancestor, which the project's own rules say must never be merged in either direction. A pull request nonetheless proposes exactly that merge, and continuous integration tests the *merged* tree. The scope guard — a test that fails if another product's directory appears in the tree — therefore fires on every run, correctly.

This is a decision, not a defect: close the pull request, split the repository, or narrow the guard. The third option softens a rule written after a real incident. It has deliberately not been taken without instruction.

The deployment audit workflow is inert

A scheduled workflow was written to check the live origin nightly. It does not run and cannot be triggered: a workflow is registered with the platform only when it exists on the default branch, which this one does not. It returns "not found" when dispatched. Until the branch situation changes the audit runs locally and nowhere else. This was verified, not assumed.

The remaining specification item needs live sellers

The adversarial suite's leakage probe asks for off-platform drift measured on a real seller cohort over real weeks, proving that escrow, the credit record and loyalty pricing actually retain sellers. Synthetic data would prove nothing. It is a human gate, not code.

The ten human gates

Company registration and trademark clearance; a payment partner agreement with counsel custody sign-off; WhatsApp Business API verification; advertising accounts as agency of record; a dummy panel and the first ten sellers onboarded by voice note alone; 100 real paid orders with zero reconciliation breaks; counsel verification per island before any market goes live; a manual install on Android and iOS; speech recognition accuracy measured on live patois with a real vendor; and the leakage probe above.

9. Where a reviewer should look for what is still wrong

An honest list of where the author would look next, given more time.

1. **Other check-then-act pairs.** The ledger's locking defect was a pattern, and the pattern may exist elsewhere. Capacity, orders and identity do lock. Everything else should be read with that question in mind.
2. **Branch coverage is 74%.** Lower than statement coverage by eleven points, which means error paths are less exercised than happy paths. Error paths are where money goes missing.
3. **Every test that cannot fail.** Two were found this session. The technique that found them — asking "could this assertion ever be false?" of each test in turn — has not been applied to the whole suite.
4. **The payment adapter is mock-complete.** No real provider has been integrated. The ledger is proven against a simulation of a provider, not a provider.
5. **Security headers.** Pages ship a content type and nothing else: no content security policy, no MIME-sniffing protection.
6. **Nobody has used it.** Every behavioural claim rests on tests and drills, not on a seller in Kingston.

10. Session commit log

Every change since the deployment work began, newest first. Commit messages are written to explain why, not what, and are worth reading alongside the diff.

Commit	Subject
9c28a07	DEPLOY.md: where to look when the question is "is this working"
d9099fc	A developer console, because the cockpit cannot say whether anything works
1f5bc2f	The ledger settled one order eight times; an adversarial review caught it
8823a6f	The §5.7 adversarial suite, and the hole it found in the first run
24ab339	A gate pinned to a path that does not exist is a gate that never runs
421a5be	The audit workflow is not registered with GitHub at all; say so
6ce354b	Scope the linter and formatter to our own code, not the neighbour's
2ffea60	CI has been red for a week, and the cause was a line I added
110417f	Refresh every measured claim, because a carried-forward number is not evidence
5fc4b7b	The Dockerfile builds, and it needed no fixes
4ef2da3	Allowlist the read-only checks this build actually runs
8248fa1	A serverless boot probes the schema; it no longer races to migrate it
57d09b5	Audit the deployed origin in a real browser, closing the machine half of P36
9848181	The link is live: the database was migrated deliberately, not by a cold start
b46d04d	BUILD_STATUS: three failures to connect one database, and what each taught
67aaf83	When the database refuses, /demo names the connection it tried
61e3fe5	The database URL may arrive under a vendor's name; /demo says what it sees
29655d5	Deploy to Vercel: the app finds its data and seeds itself at boot

11. Documents in the repository a reviewer will want

File	What it is
CLAUDE.md	The standing rules: product laws, money rules, data rules, scope law
SYCAMORE_BUILD.md	The engineering specification, including the stress-test master plan
SYCAMORE_PROMPTS.md	The 37-prompt build chain with a gate on each
BUILD_STATUS.md	The running record, including every audit and every failure on the way
DEPLOY.md	How the deployment works and the three things about it that have each gone wrong once
DEMO.md	How to run a seeded market locally and click through it

Prepared for independent review. Every claim is reproducible with the commands in section 2. Where a claim could not be verified, this document says so rather than omitting it.